

Cold-Start Bus Stop Demand Prediction

Inductive GATv2 vs. Tabular Baselines under Strict Spatial Cross-Validation

Complete Technical Report — Pipeline, Code, and Results

CE902 MSc Dissertation Project
University of Essex

Generated 19 July 2026

Abstract

This report documents the complete experimental pipeline for predicting weekday boarding demand at 17,943 London bus stops with no historical passenger data at test stops (a cold-start problem), comparing an inductive Graph Attention Network (GATv2) against seven tabular and geostatistical baselines under 33-fold leave-borough-out spatial cross-validation. It covers: the full data pipeline (raw TfL/ONS/OSM sources through to model-ready features), every model architecture implemented (with code), all eight experiment sets and their statistically-tested results, the file inventory of the project, and an honest account of what remains open. It is written to be read alongside the project's codebase, not as a replacement for it — code excerpts are illustrative, and the full source is the authoritative reference.

1 *

Contents

1 *	2
2 Project Overview	3
2.1 Research question	3
2.2 Why this matters	3
2.3 Headline finding	3
2.4 Metric and validation design	3
3 Data Pipeline	4
3.1 Overview	4
3.2 Target construction	4
3.3 Feature groups	4
3.4 The service_coverage feature: exact source and cold-start validity	4
3.5 Why leave-borough-out CV is mandatory, not a stylistic choice	6
4 Model Architectures	7
4.1 GATv2Model — the core graph model	7
4.2 MLPModel — the graph-free ablation, and eventual winner	7
4.3 The multigraph: KNN + route connectivity	7
4.4 IDW — geostatistical baseline (no learning, no features)	8
4.5 Functional-similarity edges and the Zheng et al. (2025) fusion architecture	8
4.6 The pre-registered gated-mixing model	8
5 Complete Results	10
5.1 Consolidated table	10
5.2 Experiment timeline	10
5.3 Statistical significance testing	10
5.4 Mechanism: why does GATv2 lose? (quantitatively checked, not just asserted)	11
5.5 Experiment Set 7 in detail: does a second graph type help?	12
5.6 Experiment Set 8 in detail: the pre-registered gated-mixing test	12
5.6.1 Follow-up: is this just ensembling?	12
6 External Validity: Independent-Dataset Evidence	14
7 File Inventory and Reproducibility	15
7.1 Core pipeline scripts (run in order, only if rebuilding from raw data)	15
7.2 Modelling and analysis scripts	15
7.3 What to submit / show	15
7.4 Reproducing the headline result	15
7.5 Google Colab notebook	15
8 Recommendations for the Dissertation Write-Up	17

2 Project Overview

2.1 Research question

Can weekday bus-stop boarding demand be predicted for a stop with *zero* historical passenger data, using only publicly available spatial features (accessibility, land use, planned service supply)? This is the cold-start problem, framed here as an inductive learning task: a model trained on stops in 32 London boroughs must predict demand at stops in a 33rd, entirely unseen, borough.

2.2 Why this matters

Conventional demand models (historical averages, time-series methods) cannot be applied to a stop that has not opened yet. Transport planners need demand estimates for proposed new stops before they exist. This is analogous to the cold-start problem in recommender systems (new user/item with no interaction history), but has been comparatively little explored for transit demand at the level of rigor this project applies (strict spatial cross-validation, not random splits).

2.3 Headline finding

The best model is a plain 3-layer MLP (WMAPE = 0.6311), not the graph neural network. GATv2, in its plain form, loses to every tabular baseline at $p < 0.00001$ (Wilcoxon signed-rank, paired by borough). This survives five independent attempts to help the graph: doubling the neighbourhood size ($K=10$), removing feature multicollinearity (PCA), adding a second graph based on functional similarity, and replicating a published dual-graph fusion architecture (Zheng et al., 2025). A sixth, pre-registered attempt — a learned per-node gate that lets the model freely choose how much to trust its neighbours — produced a small, statistically significant edge over the MLP, but a rigorous follow-up check found this edge is smaller than ordinary neural-network training variance, so it is reported as suggestive, not settled. Section 5 gives the complete, honest account.

2.4 Metric and validation design

WMAPE (Weighted Mean Absolute Percentage Error) is used throughout:

$$\text{WMAPE} = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i |y_i|}$$

Unlike MAPE, WMAPE does not blow up at zero-demand stops (287 of 17,943 stops, 1.6%, have zero recorded weekday boardings). Lower is better; 1.0 means the model’s total absolute error equals the total demand being predicted.

33-fold leave-borough-out cross-validation: in each fold, every stop in one of London’s 33 boroughs is withheld entirely — the model never observes its features *during a labelled training pass*, and for the graph model, the held-out stops are added to the graph only at inference time, aggregating from training-borough neighbours. This is deliberately much stricter than a random split. A concrete, dataset-verified justification for why this is necessary is given in Section 3.5.

3 Data Pipeline

3.1 Overview

Script	Produces	Source
step1_aggregate_busto.py	busto_stop_level_boardings.csv	TfL BUSTO 2023/24
step2_join_coordinates.py	stops_with_coords.csv	TfL Bus_Stops.csv
step3_lsoa_features.py	stops_features.csv	postcodes.io + AI23
step3b_osm_features.py	+OSM POI columns	OpenStreetMap (osmnx)
step3c_add_scenic.py	+poi_scenic column	OpenStreetMap
step3d_add_service_coverage.py	+service_coverage column	BUSTO row structure

3.2 Target construction

BUSTO provides survey-weighted average boardings per stop, per route, per direction, per 15-minute time slot, for a typical 2023/24 weekday. The target is the simple sum across all slots for a stop:

```
stops = (  
    raw.groupby("STOPCODE")  
        .agg(  
            stop_name=("STOPNAME", "first"),  
            total_boardings=("Boardings", "sum"),  
            n_route_dir_qhr_rows=("Boardings", "size"), # -> service_coverage later  
        )  
    .reset_index()  
)
```

Note `n_route_dir_qhr_rows` is captured here as a byproduct (a row count, not a boardings-derived value) and reused in step 3d as the `service_coverage` feature — see Section 3.4 for why this is a supply-side, not demand-side, quantity.

3.3 Feature groups

Group	Source	N	Description
AI23 accessibility	ONS/UBDC	8	30-min public-transport accessibility to employment, hospitals, GP practices, supermarkets, pharmacies, primary schools, secondary schools, built-up areas (LSOA-level)
OSM POI	OpenStreetMap	6	Counts within a 500m buffer: residential, shopping, company, education, entertainment, scenic/cultural
Service coverage	BUSTO (this study)	1	Distinct (route×direction× quarter-hour) combinations serving the stop — see 3.4
Coordinates	TfL	2	lat, lon — essential; see 3.5 for why

All non-coordinate features are `log1p`-transformed before scaling (raw skew ranged from 1.7 to 6.3 across features).

3.4 The `service_coverage` feature: exact source and cold-start validity

Formula: count of distinct (ROUTE, DIRECTION, quarter-hour) rows recorded for a stop in the BUSTO extract — `groupby("STOPCODE").agg(Boardings="size")`. It does not filter on boardings being non-zero (confirmed: all 287 zero-boardings stops have `service_coverage` ≥ 2).

Precise provenance statement (important — avoid overclaiming in the write-up): this is neither literally “GTFS-planned service” nor “realised service” in the sense of observed ridership. It was computed from BUSTO’s row structure (a supply/schedule-side count: how many distinct scheduled service patterns touch this stop), not downloaded from a GTFS feed. Its cold-start validity rests on a *forward-looking argument*: an equivalent count is derivable from a GTFS `stop_times.txt` before a genuinely new stop opens, since it is a property of the timetable, not of observed demand. This distinction should be stated explicitly and precisely in the methodology chapter.

Impact: adding this single feature was the single largest driver of improvement in the entire project — WMAPE fell from ~ 0.80 to ~ 0.63 for every tabular model (Experiment Set 3; see the timeline in Section 5).

3.5 Why leave-borough-out CV is mandatory, not a stylistic choice

AI23 features are defined at LSOA (Lower Layer Super Output Area) resolution — multiple bus stops within the same LSOA receive an *identical* 8-dimensional feature vector. This was verified concretely on a real example: STOPCODE 1000 (Westminster Station / Parliament Square, LSOA E01004733) shares its LSOA with 12 other stops spanning 1,109 metres. All 13 have exactly one distinct AI23 vector among them.

More importantly for the graph model: of stop 1000’s actual $K=5$ nearest neighbours in the constructed KNN graph, **3 of 5 share its LSOA** (and therefore its exact AI23 vector). Under a random stop-level split, GATv2’s message-passing step would aggregate directly from labelled near-duplicates at inference — not generalising from environmental features, but substantially re-constructing the answer from a labelled twin.

Why leave-borough-out CV eliminates this: LSOAs nest within London boroughs almost exactly — verified empirically on this dataset: 4,219 of 4,222 unique LSOAs (99.93%) map to exactly one borough. (The 3 exceptions were traced precisely: `lad_name` and `lsOA11cd` are assigned independently per stop via a postcode lookup, so a stop whose postcode sits immediately at a borough boundary can occasionally receive an inconsistent label; cross-checked against the ONS LSOA-to-LAD reference table, the LSOA’s official borough always matches the majority of its constituent stops — e.g. LSOA E01002563, officially named “Hounslow 006A”, has 6 of 7 stops correctly labelled Hounslow.) Because LSOAs essentially never split across the leave-one-borough-out fold boundary, no test stop can share an LSOA — or therefore an identical AI23 vector — with any training stop.

4 Model Architectures

All models share: $\log_{1p}(\text{boardings})$ target, Huber loss ($\delta = 0.5$, robust to the residual heavy tail after log transform), Adam optimizer, a stratified (5 demand-quantile) internal validation split for early stopping, and a per-fold `StandardScaler` fitted on training stops only. Hyperparameters (`HIDDEN_DIM=64`, `HEADS=4`, `DROPOUT=0.15`, `LR=5e-4`) are fixed across every experiment — no hyperparameter search was performed anywhere in this project.

4.1 GATv2Model — the core graph model

```
class GATv2Model(torch.nn.Module):
    def __init__(self, in_ch):
        self.c1 = GATv2Conv(in_ch, 64, heads=4, dropout=0.15, concat=True)
        self.c2 = GATv2Conv(256, 1, heads=1, dropout=0.15, concat=False)
        self.skip = torch.nn.Linear(in_ch, 256, bias=False) # residual -- critical

    def forward(self, x, edge_index):
        h = F.elu(self.c1(x, edge_index)) + self.skip(x) # graph + residual
        h = F.dropout(h, p=0.15, training=self.training)
        return self.c2(h, edge_index).squeeze(-1)
```

The residual skip connection (Bug 5 in the project’s bug log) was found to be *architecturally critical*: without it, all information is forced through graph aggregation even when the neighbourhood is pure noise, which happens systematically at test time under leave-borough-out CV (a held-out stop’s only neighbours are in adjacent, different boroughs). With the skip, the model can learn to bypass the graph per node when it is unhelpful — but empirically (Section 5) it still under-uses this escape hatch relative to a pure MLP.

4.2 MLPModel — the graph-free ablation, and eventual winner

```
class MLPModel(torch.nn.Module):
    def __init__(self, in_ch):
        h = 256
        self.net = torch.nn.Sequential(
            torch.nn.Linear(in_ch, h), torch.nn.ELU(), torch.nn.Dropout(0.15),
            torch.nn.Linear(h, h // 2), torch.nn.ELU(), torch.nn.Dropout(0.15),
            torch.nn.Linear(h // 2, 1),
        )
    def forward(self, x, edge_index=None):
        return self.net(x).squeeze(-1)
```

Identical capacity to GATv2Model’s hidden path, with zero message passing. This is the cleanest possible ablation: same features, same training protocol, same capacity, only the graph is removed.

4.3 The multigraph: KNN + route connectivity

Two edge types are combined into a single graph for the plain GATv2 model:

- **Geographic KNN** ($K=5$, Haversine distance): captures spatial proximity.
- **Route connectivity**: consecutive stops on the same bus route and direction (from BUSTO), bidirectional. Rationale: stops on the same route share passenger flow driven by the same vehicles, a functionally meaningful edge geography alone cannot capture.

Combined: 124,234 directed edges over 17,943 nodes (deduplicated union).

4.4 IDW — geostatistical baseline (no learning, no features)

Motivated by Liu et al. (2017)’s NYC Citi Bike cold-start work (gravity models + spatial interpolation). Predicts a held-out stop’s $\log_{10}(\text{boardings})$ as an inverse-distance-weighted average of its $k=20$ nearest *training* stops’ $\log_{10}(\text{boardings})$ — lat/lon only, no AI23/OSM/service_coverage features at all:

```
def idw_predict(train_lat, train_lon, train_y_log, test_lat, test_lon, k=20, power=2):
    dist, idx = BallTree(train_coords, metric="haversine").query(test_coords, k=k)
    w = 1.0 / np.power(dist_km + 1e-6, power)
    return np.sum(w * train_y_log[idx], axis=1) / np.sum(w, axis=1)
```

Result: WMAPE = 0.8487 — better than the naive historical average (1.0822) but worse than every feature-based model. This establishes that pure geographic proximity carries real but limited signal, sharpening the interpretation of the graph model’s results.

4.5 Functional-similarity edges and the Zheng et al. (2025) fusion architecture

Zheng et al. (2025) build a second graph, G_2 , from Pearson correlation of each stop’s POI count vector (not historical demand, as an earlier version of this project’s own documentation had mistakenly assumed — corrected 17 July 2026 after re-reading the source equations), connecting pairs with $\rho > 0.8$.

A necessary recalibration, discovered and documented, not silently applied: the literal $\rho > 0.8$ threshold on this project’s 6 OSM POI categories produces a near-complete graph — 48,087,084 directed edges, average degree 2,680 (vs. $\sim 124k$ edges, degree ~ 7 , for the existing graph). This reflects real zero-inflation in the POI counts (many stops share “mostly zero except one category” profiles). A threshold sweep found $\rho > 0.99$ combined with a top-10-per-stop cap gives a comparably sparse graph (133,646 edges, degree ~ 7.4) to the existing one.

Two implementations were tested (Experiment Set 7, Section 5.5):

1. Functional-similarity edges mixed into the *same* adjacency matrix as KNN+route (one GATv2, three edge types).
2. A faithful replica of Zheng’s actual architecture: *separate* GATv2 branches over the geographic graph G_1 and the functional-similarity graph G_2 , concatenated, then a third GATv2 pass over the union graph $G_3 = G_1 \cup G_2$, plus a residual skip (not in the original paper, added for fair comparison against this project’s own GATv2Model):

```
class ZhengFusionGAT(torch.nn.Module):
    def forward(self, x, ei_g1, ei_g2, ei_g3):
        z_adj = F.elu(self.gat_g1(x, ei_g1))
        z_f = F.elu(self.gat_g2(x, ei_g2))
        z_fused = torch.cat([z_adj, z_f], dim=1)
        return (self.gat_g3(z_fused, ei_g3) + self.skip(x)).squeeze(-1)
```

4.6 The pre-registered gated-mixing model

The final architectural experiment (Experiment Set 8) tested a specific hypothesis: message passing dilutes a strong node-local signal (motivating evidence: the MLP–GATv2 gap is 0.75pp *without* service_coverage and 8.76pp *with* it — the graph looks worse specifically when the node-local signal is strongest). An architecture that can freely choose how much neighbour information to use should, if the hypothesis is true, learn to use almost none:

```
class GatedGATv2Model(torch.nn.Module):
    def __init__(self, in_ch):
        self.mlp = MLPModel(in_ch) # UNMODIFIED
        self.gat = GATv2Model(in_ch) # UNMODIFIED, own residual skip intact
        self.gate = torch.nn.Linear(in_ch, 1)
```



```

def forward(self, x, edge_index):
    mlp_out = self.mlp(x)
    gat_out = self.gat(x, edge_index)
    alpha = torch.sigmoid(self.gate(x)).squeeze(-1)
    return alpha * mlp_out + (1 - alpha) * gat_out, alpha

```

MLPModel and GATv2Model are used completely unmodified, so that $\alpha=1$ recovers the standalone MLP exactly and $\alpha=0$ recovers the standalone GATv2 exactly — verified directly with `torch.allclose` before running anything, not merely asserted by construction. The only new parameter is a single `Linear(in_ch, 1)` gate. This was a pre-registered, one-attempt experiment: three outcome buckets were fixed in writing *before* training, with the interpretation to be applied regardless of which bucket the result landed in (Section 5.6).

5 Complete Results

5.1 Consolidated table

Mean WMAPE, 33-fold leave-borough-out CV. Best value in each column is the practical headline; the AI23+OSM+SC column is the frozen “headline” feature set.

Model	AI23-only	OSM-only	AI23+OSM	AI23+OSM+SC	PCA(AI23)+OSM+S
HistAvg	1.0822	1.0822	1.0822	1.0822	1.0822
IDW	0.8487	0.8487	0.8487	0.8487	0.8487
MLR (Ridge)	0.8120	0.8078	0.7982	0.6404	0.6401
RF	0.8128	0.8064	0.7970	0.6428	0.6400
XGBoost	0.8207	0.8084	0.8075	0.6437	0.6474
MLP	0.8114	0.8058	0.7983	0.6311	0.6299
GATv2 (plain)	0.8241	0.8259	0.8058	0.7187	0.7171
GATv2, K=10 fairness	—	—	—	0.7372	—
GATv2, func-sim edges	—	—	—	0.7352	—
GATv2-Fusion (G1/G2/G3)	—	—	—	0.7070	—
Gated (learned α , 3-seed)	—	—	—	0.6277 [†]	—

[†] See Section 5.6 — statistically significant vs. MLP but the effect is smaller than observed MLP training variance; reported as suggestive, not settled.

5.2 Experiment timeline

v1 (19 Jun) First baseline suite, 3 feature sets $\times$ 4 models. Had a critical bug (validation loss computed from a dropout-active forward pass, corrupting early stopping for every neural model). Best: RF/AI23+OSM = 0.7975.

v2 (22 Jun) 5 bugs fixed (including the critical one) plus the residual skip connection added to GATv2. Results never isolated to a standalone file — superseded before being saved separately.

v3 (code 22 Jun, executed 16–17 Jul) Stratified validation split, Huber loss, faster training; service_coverage engineered but not run at full scale until v4.

v4 (16–17 Jul) service_coverage tested — WMAPE fell from ~ 0.80 to ~ 0.63 . MLR and XGBoost baselines added. K=10 fairness check (GATv2 got worse, not better). IDW baseline added. PCA multicollinearity test (null result). Optional V/C overcrowding experiment run as a secondary analysis.

Experiment Set 7 (17–18 Jul) Functional-similarity edges: single-graph variant worse (0.7352); full G1/G2/G3 fusion architecture better (0.7070) but not statistically significant on its own.

Experiment Set 8 (18–19 Jul) Pre-registered gated-mixing test and its residual- correlation follow-up (Section 5.6).

5.3 Statistical significance testing

Paired Wilcoxon signed-rank tests across the 33 boroughs (same boroughs scored under every model — a matched-pairs design):

A	B	mean(A−B)	p-value	Significant?
GATv2	RF	+0.0759	< 0.00001	Yes (GATv2 worse)
GATv2	MLP	+0.0875	< 0.00001	Yes (GATv2 worse)
GATv2	MLR	+0.0783	< 0.00001	Yes (GATv2 worse)
MLP	RF	−0.0117	0.00225	Yes (MLP better)
MLP	MLR	−0.0092	0.00081	Yes (MLP better)
RF	MLR	+0.0024	0.357	No (tied)
MLP	XGBoost	−0.0126	0.060	No (borderline, tied)
RF	XGBoost	−0.0009	0.386	No (tied)
GATv2-Fusion	GATv2 (plain)	−0.0117	0.126	No
GATv2-Fusion	MLP	+0.0759	< 0.00001	Yes (Fusion worse)
Gated	MLP	−0.0035	0.00697	Yes [†] (Gated better)
Gated	GATv2 (plain)	−0.0910	< 0.00001	Yes (Gated better)

[†] survives Holm-Bonferroni correction across the 2-test family, but see Section 5.6 for the variance caveat that qualifies this specific result.

The headline takeaway: GATv2 loses to tabular models at $p < 0.00001$ across every plain and moderately-modified variant tested. “MLP is best” is significant against RF and MLR specifically, but MLP and XGBoost are statistically tied — report them as jointly best, not MLP alone as unambiguous winner.

5.4 Mechanism: why does GATv2 lose? (quantitatively checked, not just asserted)

Three candidate explanations were tested using only data already on hand (the deterministic KNN graph structure plus existing per-fold results — no new model training):

Hypothesis	Pearson r	Spearman ρ	Robust?
Local smoothing (% KNN neighbours in a different borough)	0.523 ($p=0.0018$)	0.439 ($p=0.0105$)	Yes — both sig
Sample-size artifact (n_{test} vs. gap)	−0.312 ($p=0.077$)	−0.170 ($p=0.344$)	No — fails Spearman
Geographic scale (bounding-box diagonal, km)	−0.429 ($p=0.013$)	−0.218 ($p=0.222$)	No — fails Spearman

Only the local-smoothing / cross-borough-contamination hypothesis survives both a parametric and a rank-based test. Boroughs whose stops have a higher fraction of their K=5 nearest neighbours in an *adjacent* borough show a systematically larger GATv2-vs-RF gap. The other two candidate explanations show an apparent linear correlation but do not survive the more robust rank-based test, consistent with a small number of outlier boroughs (City of London: $n=101$, tiny area, largest gap) driving the Pearson result by leverage rather than a genuine, consistent effect.

5.5 Experiment Set 7 in detail: does a second graph type help?

Variant	WMAPE
GATv2 (plain, reference)	0.7187
GATv2 + functional-similarity edges, single graph	0.7352 (<i>worse</i>)
GATv2-Fusion, separate G1/G2 branches (Zheng-style)	0.7070 (<i>better</i>)

Interpretation: mixing a second edge type into the *same* adjacency matrix as geography made things worse — plausibly because a single attention mechanism cannot distinguish “nearby” from “functionally similar,” conflating two different relationship types into one set of attention weights. Processing them through *separate* branches before fusing avoided that conflation and was the single most effective graph intervention in the project. It still lost to MLP (7.59pp gap, narrower than plain GATv2’s 8.76pp but not closed) and the improvement over plain GATv2 itself did not reach significance in isolation (see Section 5, $p = 0.126$).

Per-borough nuance: under Fusion, Hillingdon (the peripheral borough singled out in early drafts as the “worst case”) improved to a near-tie with RF (0.7663 vs. 0.7647), while Camden (previously fine) became the single largest gap (0.7412 vs. RF’s 0.5230). The identity of “which borough is hardest” is sensitive to the specific architecture, not a fixed property of geography — borough examples in the discussion chapter should always be pulled fresh from the run being discussed, not reused across configurations.

5.6 Experiment Set 8 in detail: the pre-registered gated-mixing test

Pre-registration (fixed before running anything): three outcome buckets were written in advance in `step4f_gated_mixing.py`’s docstring. Result summary:

Metric	Value
Gated WMAPE, 3-seed mean (seeds 42/142/242)	0.6277 (sd across seeds 0.0019)
vs. MLP (0.6311)	−0.35pp, $p = 0.00697$ (significant)
vs. GATv2 plain (0.7187)	−9.10pp, $p < 0.00001$ (significant)
Mean learned α	0.5212 (median 0.520, IQR [0.456, 0.587])
Nodes with $\alpha > 0.9$ (“ignores neighbours”)	0.00%
Nodes with $\alpha < 0.1$ (“pure message passing”)	0.00%
Corr(per-borough mean α , per-borough gap vs. MLP)	$r = -0.026$, $p = 0.887$ (none)

Literal trigger vs. what the α evidence supports (both stated, not reconciled into one story): on the letter of the pre-registered rule, this triggers the “Finding 3 must be rewritten” bucket. But the mechanism that framing implies — a gate that learned *when* the graph is useful and adapted accordingly — is not what the α statistics show. α never saturates, is uncorrelated with where the graph helps or hurts, and varies more across random seeds (std 0.085) than across nodes (std 0.101 pooled) — i.e. most of its apparent variation is training noise, not a stable per-node signal.

5.6.1 Follow-up: is this just ensembling?

A retrain of standalone MLP and GATv2 (identical seed, folds, architecture; only addition: saving raw per-stop predictions, which the original runs did not do) tested this directly.

Quantity	Value
Residual correlation, MLP vs. GATv2 (log1p-scale)	$r = 0.9035$ ($p \approx 0$)
Residual correlation (original-scale)	$r = 0.9280$ ($p \approx 0$)
Standalone MLP WMAPE (this retrain)	0.6248
Standalone GATv2 WMAPE (this retrain)	0.7311
Naive fixed 50/50 blend (no learned gate)	0.6655 — <i>worse</i> than MLP

Ensembling is ruled out. Residuals are highly correlated, not decorrelated, and a naive fixed blend of the same two model types is clearly worse than MLP alone, not better — the opposite of what a variance-reduction/ensembling explanation would predict. Whatever the *learned* gate achieves must come from the joint, end-to-end co-adaptation of the two branches during training, not from passive averaging of independently-trained models.

But an important caveat surfaced unprompted: this retrain’s standalone MLP scored 0.6248, not the frozen headline’s 0.6311 — a **0.63pp swing from ordinary training stochasticity on the identical seed and configuration**, nearly double the entire Gated-vs-MLP effect size being investigated (0.35pp). The 3-seed test controlled for the Gated model’s own variance, but compared it against a single stored MLP number whose own run-to-run stability was, until this check, unverified.

Final, defensible written claim (use this, not a simplified version): “Forced 100%-or-0% graph/non-graph mixing is demonstrably the wrong architectural choice, with a large and robust effect (9.10pp, $p < 0.00001$, confirmed via a ruled-out ensembling alternative). Whether adaptive, jointly-trained mixing yields a genuine additional edge over the best tabular model, beyond ordinary training noise, is suggestive but not established beyond reasonable doubt — a proper multi-seed baseline for standalone MLP is the natural next check, not performed here to respect the project’s one-attempt-per-experiment discipline.”

6 External Validity: Independent-Dataset Evidence

A full second-dataset empirical replication was assessed as disproportionate scope this late in the project. Instead, two already-published, fully independent studies were formalised as convergent evidence for the *outcome* (not a replication of the *method*, since neither uses cold-start spatial CV or a graph architecture):

- **Yusuf et al. (2025)** — stop-level bus demand prediction in Trondheim, Norway (an entirely independent transit system, country, and dataset). Same task granularity, same *kind* of features (land-use/demographic buffer variables). Found tree-based ML (XGBoost) beats deep learning, and independently diagnosed the cause as a lack of inductive bias for transit network spatial structure — closely anticipating this project’s cross-borough-contamination mechanism (Section 5.4).
- **Grinsztajn et al. (2022)** — a domain-general benchmark (NeurIPS 2022, ~45 tabular datasets, no transit connection) finding tree-based methods consistently beat deep learning on tabular data, for reasons (irregular target functions, sensitivity to uninformative features, unhelpful rotational invariance) that apply directly to this project’s feature space.

What remains this project’s own contribution: the cold-start framing, the spatial-CV-induced mechanism, and its confirmation across five independent graph interventions with $p < 0.00001$ each time.

7 File Inventory and Reproducibility

7.1 Core pipeline scripts (run in order, only if rebuilding from raw data)

step1_aggregate_busto.py → step2_join_coordinates.py → step3_lsoa_features.py → step3b_osm_features.py → step3c_add_scenic.py → step3d_add_service_coverage.py.

7.2 Modelling and analysis scripts

Script	Purpose
step4_model.py	Main script: HistAvg/IDW/MLR/RF/XGBoost/MLP/GATv2, 33-fold CV. Flags: <code>--ai23-only</code> , <code>--osm-only</code> , <code>--with-sc</code> , <code>--k10</code> , <code>--pca-ai23</code> , <code>--func-sim</code> , <code>--quick</code> .
step4c_fast_baselines.py	HistAvg/MLR/RF/XGBoost only (no NN), for the 3 pre-SC feature sets, without redoing expensive GATv2/MLP training.
step4d_idw_baseline.py	IDW baseline (Section 4.4).
step4e_zheng_fusion.py	Zheng-style G1/G2/G3 fusion GAT (Section 4.5).
step4f_gated_mixing.py	Pre-registered gated-mixing test (Section 4.6).
step4g_gated_analysis.py	Statistics for Experiment Set 8: Wilcoxon, Holm-Bonferroni, α distribution, per-borough correlation.
step4h_residual_correlation.py	The ensembling-vs-signal follow-up (Section 5.6).
step5a_borough_map.py	Choropleth map, all 33 boroughs.
step5b_vc_experiment.py	Secondary V/C (overcrowding) target experiment.
step6_consolidated_table.py	Builds the master results table.
step7_borough_extract.py	Per-borough extract for the discussion chapter (takes a model-column argument — do not assume the default “GATv2” column when a CSV carries a differently-named variant).

7.3 What to submit / show

Essential: every `step*.py` script, `experiment_log.md` (the master technical record), `RUNBOOK.md`, `consolidated_results_table.csv`, all `borough_extract*.csv`, `borough_wmape_map.png`, `dissertation_colab.ipynb`, and every non-`quick` `results_cv_*/results_summary_*.csv` file (this is the evidence base for every claim in this report).

Safe to delete (confirmed duplicates or smoke-test scratch, verified before recommending): `results_cv_multigraph.csv` and `results_summary_multigraph.csv` (byte-identical to `*_func_sim.csv`), all `*_quick.csv` result files, `results_cv_colab.csv/results_summary_colab.csv/wmape_per_borough.png` (byproducts of a local notebook-verification run, not project results), `_pycache_/_/`.

Needs a decision, not assumed: two dissertation LaTeX drafts exist (`dissertation.tex`, 876 lines, and `dissertation_main.tex`, 301 lines, in different folders) — confirm which is current before writing further. `build_excel.py` is broken (points at already-deleted files) and stale (predates most of this project’s results).

7.4 Reproducing the headline result

```
python step3d_add_service_coverage.py
python step4_model.py --with-sc          # -> results_cv/summary_multigraph.csv
# rename to results_cv_ai23_osm_sc.csv / results_summary_ai23_osm_sc.csv
```

Expected runtime: 1.5–2.5 hours, CPU-only (no GPU used anywhere in this project’s local runs; the Colab notebook supports GPU and will be substantially faster).

7.5 Google Colab notebook

`dissertation_colab.ipynb` mirrors the current 7-model pipeline exactly (verified by locally executing the extracted notebook code before publishing it as done). To use: upload `stops_features_osm.csv`

and `route_edges.csv` to `MyDrive/dissertation/` in Google Drive, open in Colab, run top to bottom. **On portability to another dataset:** the methodology (leave-region-out CV, the model suite, the training protocol) is fully transferable to any spatial cold-start problem. The code is not literally plug-and-play — it requires rebuilding the equivalent of steps 1–3d for a new data source (the real work, since AI23 is UK-specific) and updating ~ 10 –15 lines of column-name constants. The public NYC Citi Bike dataset (already in this project’s literature review via Liu et al., 2017) would be the most natural next test case.

8 Recommendations for the Dissertation Write-Up

1. **Lead Chapter 4 with the consolidated table and the statistical significance table together**, not the WMAPE numbers alone — the p -values are what make the central claim defensible under viva questioning.
2. **State the service_coverage provenance precisely** (Section 3.4): it was computed from BUSTO, and its cold-start validity is a forward-looking argument about GTFS-derivability, not a description of how it was actually sourced here. Getting this distinction wrong is the single most likely place for an examiner to find a gap.
3. **Use the LSOA/leakage argument (Section 3.5) explicitly in the methodology chapter**, not just a citation to Roberts (2017) — it is a concrete, dataset-specific, checkable mechanism, not an abstract appeal to spatial autocorrelation.
4. **Do not reuse a single borough example (e.g. the old Camden/Hillingdon anecdote) across different model configurations** — the per-borough picture changes with the architecture (Section 5.5). Pull examples fresh from whichever run is being discussed.
5. **Report Experiment Set 8 with both halves stated**, per Section 5.6’s final claim wording. Do not simplify to either “GATv2 wins after all” or “it’s just noise” — both would misrepresent what was actually established.
6. **Decide the fate of the exploratory experiments** (PCA variant, func-sim, Fusion, Gated) before finalising Chapter 4’s structure: main comparison table, or appendix? All are fully documented in `experiment_log.md` either way.
7. **Resolve the two-draft discrepancy** in the LaTeX sources before continuing to write, to avoid divergent edits.